

Міністерство освіти і науки України
Національний університет «Запорізька політехніка»

кафедра програмних засобів

ЗВІТ

з лабораторної роботи № 4

з дисципліни «Технології розподілених систем та паралельних
обчислень» на тему:

«ФУНКЦІЇ ОБМІНУ В MPI»

Виконав:

ст. гр. КНТ-113сп

Іван ЩЕДРОВСЬКИЙ

Прийняв:

Старший викладач

Олександр КАЧАН

1 Мета роботи

Продовжити вивчення функцій обміну бібліотеки MPI, зокрема, функцій колективного обміну. Освоїти деякі прийоми їх використання для розподілу даних та обчислень між паралельними процесорами

2 Завдання до лабораторної роботи

2.1 Програма виконує паралельне обчислення суми ряду $\sum_{i=0}^{N-1} a_i x_i$

Для цього до кожного з P процесів за допомогою функції `MPI_Scatter()` передається відповідна частина масивів чисел a_i та x_i , кожна з яких складається з $M = \frac{N}{P}$ елементів. Процеси обчислюють часткові суми ряду (змінна `sum`), які далі за допомогою функції `MPI_Reduce()` приводяться до загальної суми (змінна `total`).

2.1.1 Другий варіант: $\sum_i a_i \cdot \cos(x_i)$, де $i = 0, 1, \dots, N$; $N = 240000$; $x_i = 0.0001 * i$; a_i – випадкові дійсні числа в діапазоні $(-1; 1)$. Функцію $\cos(x_i)$ обчислювати за допомогою розкладання її в ряд Тейлора: $\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \dots + (-1)^n \frac{x^{2n}}{(2*n)!} + \dots$.

2.1.2 Виміряйте час виконання розробленої програми на одному, двох, трьох та чотирьох процесорах для $N=240000$ та визначте прискорення обчислень для цих випадків. Дані занесіть до таблиці 4.1 (форми таблиць наведені нижче). Намалюйте графік залежності прискорення від кількості процесорів.

2.1.3 Дослідити та намалювати графіки залежності прискорення від кількості членів ряду при обчисленнях на чотирьох процесорах, змінюючи кількість членів ряду від 20000 до 240000 з дискретністю 20000. Вимірювання виконуйте 3 рази з інтервалом між вимірюваннями 15 хвилин, результати вимірювань занесіть до таблиці 4.2.

2.1.4 Додайте до програми код для вимірювання часу, який витрачається на виконання двох функцій `MPI_Scatter()`, дослідіть та намалюйте графіки залежності

цього часу від довжини масиву, що розсилається при виконанні задачі на чотирьох процесорах. Результати занесіть до таблиці 4.3

2.2 Нехай необхідно обчислити суму ряду $\sum_{i=1}^N a_i x_i^i$, де N - досить велике число та обчислення слід розподілити між P процесорами, причому, степінь чисел x_i повинна обчислюватись шляхом їх перемноження. В програмі реалізовано рівномірний розподіл навантаження обчислень.

2.2.1 Для перевірки результату паралельного обчислення додайте до програми код, який обчислює суму даного ряду тільки в нульовому процесі та використовує для обчислення степеня бібліотечну функцію.

2.2.2 Використовуючи наведені вище програми, розробіть власну програму, в якій між паралельними процесорами розподіляється однакова кількість членів ряду $\sum_{i=1}^N a_i x_i^i$.

2.2.3 Виміряйте час роботи програми для трьох, досить великих значень кількості членів ряду і порівняйте його з часом вирішення цієї задачі за допомогою вихідної програми даного завдання для двох, трьох та чотирьох процесорів. Результати наведіть у вигляді таблиці.

3 Виконання лабораторної роботи

Для виконання першого завдання було взято готовий код з методичних вказівок та виконано його рефактор, а також виправлено декілька багів. Оригінальний код не запускався правильно, оскільки при $N=400000$ ми намагаємось створити два double масиви, і вони займають забагато місця. Ця логіка була переписана на динамічні масиви.

Запуск виправленого оригінального коду показаний на рисунку 1.

```
C:\home\university-4\distributed-systems\lb4\x64\Debug>mpiexec -n 10 lb4.exe
6: Hello (p = 10)
3: Hello (p = 10)
1: Hello (p = 10)
8: Hello (p = 10)
4: Hello (p = 10)
9: Hello (p = 10)
5: Hello (p = 10)
7: Hello (p = 10)
0: Hello (p = 10)
0: t=0.004675 sec.
0: sum in 10 procs is 46115.45234
2: Hello (p = 10)
```

Рисунок 1 – Запуск оригінального коду завдання 1

Для виконання індивідуального завдання 2.1.1 за другим варіантом було взято код з завдання 1 за основу. В ньому було змінено логіку створення масиву а на рандомизацію від -1 до 1, видалено бродкастинг, оскільки ми точно знаємо N та кількість процесів в кожному процесі і так, а тому цей функціонал тільки б сповільнював роботу. Також було створено функцію для розрахунку косинусу.

На рисунку 2 показано запуск алгоритму на одному процесі, а на рисунку 3 показано запуску на двох процесях.

```
C:\home\university-4\distributed-systems\lb4\x64\Debug>mpiexec -n 1 lb4.exe
0: Hello (p = 1)
0: Generating array
0: Received 240000 elements
0: Processor sum = 235.540546
0: t=0.225586 sec.
0: sum in 1 procs is 235.54055
0: Goodbye
```

Рисунок 2 – Запуск першого індивідуального завдання на одному процесі

```
C:\home\university-4\distributed-systems\lb4\x64\Debug>mpiexec -n 2 lb4.exe
```

```
1: Hello (p = 2)
1: Received 120000 elements
1: Processor sum = -136.761735
1: Goodbye

0: Hello (p = 2)
0: Generating array
0: Received 120000 elements
0: Processor sum = -47.367752
0: t=0.122029 sec.
0: sum in 2 procs is -184.12949
0: Goodbye
```

Рисунок 3— Запуск першого індивідуального завдання на двох процесорах

Далі згідно завдання потрібно виконати заміри при запуску програми на 1, 2, 3 та 4 процесорах та визначити прискорення обчислень. N у всіх випадках 240000. Результати вимірів показані в таблиці 1.

Таблиця 1 - Залежність прискорення від кількості процесорів ($N=240000$, номер робочого місця 30)

Число проц. P	T1	Tr	S
1	0.225586	0.225586	1
2	0.225586	0.123444	1.83
3	0.225586	0.089171	2.53
4	0.225586	0.062330	3.62

На рисунку 4 показано залежність прискорення від числа процесорів.

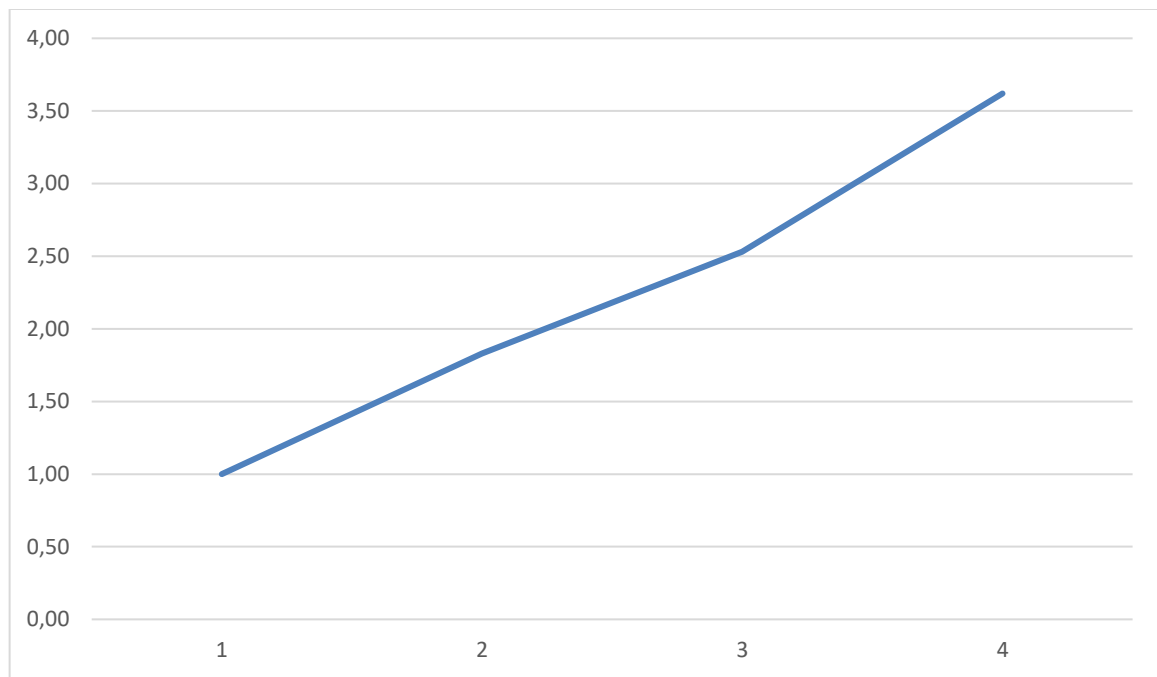


Рисунок 4 – Залежність прискорення від числа процесорів

Для виконання наступного завдання було виміряно час запуску алгоритму з $P = 1$ та $P = 4$ з N від 20000 до 240000 з кроком в 20000. Результати замірів показані на таблиці 2.

Таблиця 2 – Залежність прискорення від кількості елементів

N	T1 (сек)	T4 (сек)	S (Прискорення)
20 000	0,018036	0,007137	2,53
40 000	0,036756	0,012570	2,92
60 000	0,052727	0,019402	2,72
80 000	0,070664	0,023744	2,98
100 000	0,091099	0,030141	3,02
120 000	0,115005	0,032192	3,57
140 000	0,126695	0,040740	3,11
160 000	0,146760	0,047930	3,06
180 000	0,165723	0,053087	3,12
200 000	0,186045	0,055855	3,33
220 000	0,202874	0,066784	3,04
240 000	0,228321	0,067152	3,40

Також було побудовано графік залежності прискорення від кількості елементів, який показаний на рисунку 5.

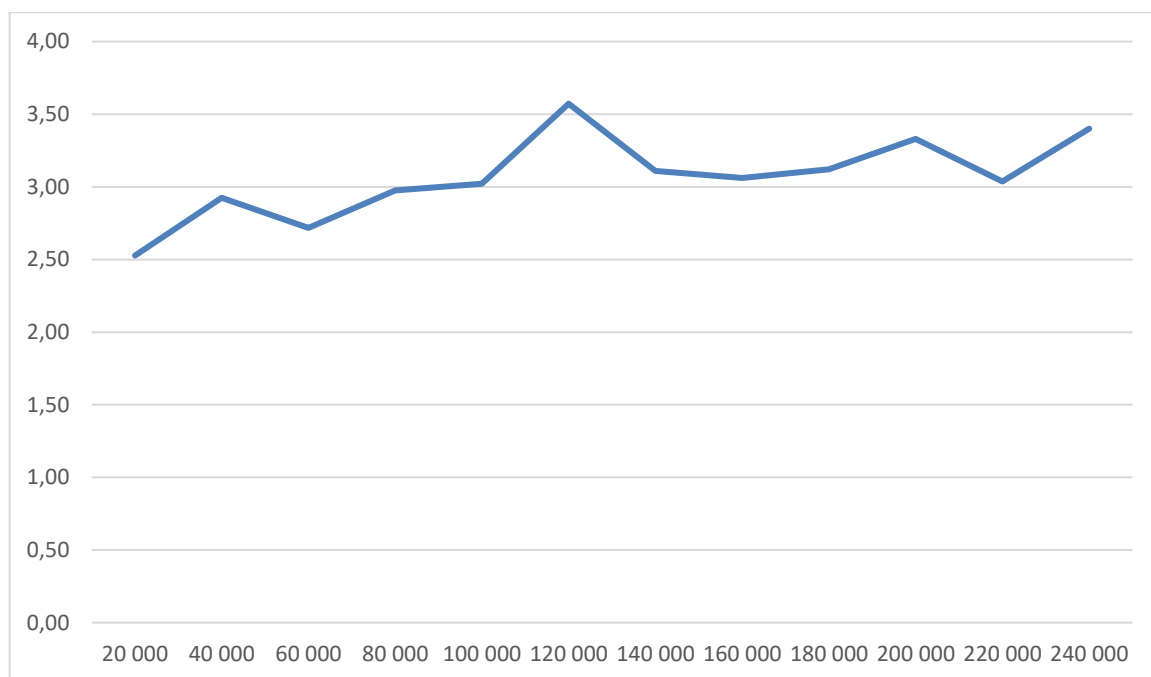


Рисунок 5 – Залежність прискорення від кількості елементів

Для виконання наступного завдання, а саме розрахунку швидкості роботи MPI_Scatter було тимчасово модифіковано код, щоб розрахувати час саме роботи передачі даних.

Важливо зауважити, що хоч по завданню треба використовувати 2 MPI_Scatter в реалізованому алгоритмі використовується тільки один, оскільки другий не є потрібним.

При вимірах було взято максимальний час процесора з одного запуску. Результати вимірів показані в таблиці 3.

Таблиця 3 – Залежність швидкості розсилання від довжини масиву при $P = 4$

N	Tscatter	S (байт/с)	S (МБ/с)
20 000	0,001583	202 147 821	192,78
40 000	0,003368	190 023 753	181,22
60 000	0,003709	258 829 873	246,84
80 000	0,004303	297 466 884	283,69
100 000	0,006086	262 898 455	250,72
120 000	0,006660	288 288 288	274,93
140 000	0,007627	293 693 457	280,09
160 000	0,008335	307 138 572	292,91
180 000	0,009798	293 937 538	280,32
200 000	0,009707	329 659 009	314,39
220 000	0,010551	333 617 667	318,16
240 000	0,011696	328 317 373	313,11

На рисунку 6 показана залежність часу виконання в секундах від кількості елементів N.

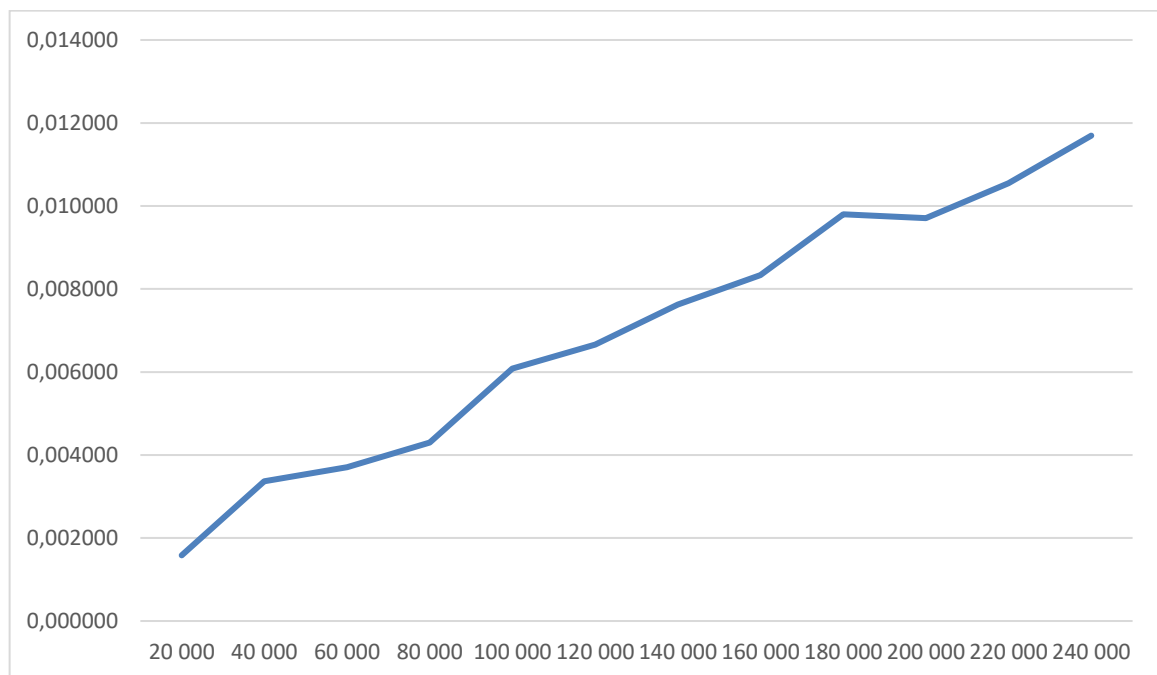


Рисунок 6 – Залежність часу виконання від кількості елементів

Для виконання завдання 2.2 було взято готовий код з методичних вказівок, виконано його рефакторинг та виправлено помилки з міграцією на C++ та використання новішої версії Visual Studio. Також до програми було додано більше логів. Результат запуску коду з завдання 2.2 показаний на рисунку 7.

```
C:\home\university-4\distributed-systems\lb4\x64\Debug>mpiexec -n 4 .\lb4.exe

3: Hello (p = 4)
3: Bye

0: Hello (p = 4)
0: Process 0 will be do 4999 multiples
0: Process 1 will be do 2071 multiples
0: Process 2 will be do 1589 multiples
0: Process 3 will be do 1341 multiples
0: Total=4.961320430501e-03
0: Time=0.260595 sec.
0: Bye

1: Hello (p = 4)
1: Bye

2: Hello (p = 4)
2: Bye
```

Рисунок 7 – Запуск коду з завдання 2.2

Для виконання завдання 2.2.1 в код для нульового рангу було додано розрахунок суми, щоб перевірити, що сума розрахована на одному процесорі та на багатьох є однаковою. В результаті перевірки сума є ідентичною, це показано на рисунку 8.

```
C:\home\university-4\distributed-systems\lb4\x64\Debug>mpiexec -n 4 .\lb4.exe

1: Hello (p = 4)
1: Bye

2: Hello (p = 4)
2: Bye

3: Hello (p = 4)
3: Bye

0: Hello (p = 4)
0: Process 0 will be do 4999 multiples
0: Process 1 will be do 2071 multiples
0: Process 2 will be do 1589 multiples
0: Process 3 will be do 1341 multiples
0: Total test=4.962169310217e-03
0: Total=4.962169310217e-03
0: Time=0.271264 sec.
0: Bye
```

Рисунок 8 – Порівняння тестової суми на одному процесорі та на багатьох

Далі, для виконання завдання 2.2.2 було створено нову функцію на основі попередньої яка розподіляє між всіма процесорами рівну кількість елементів. Останній процесор буде додатково виконувати всі елементи, які залишились після розподілення оскільки в застосунку використовується ділення `int`.

Результати запуску нового алгоритму показані на рисунку 9.

```

C:\home\university-4\distributed-systems\lb4\x64\Debug>mpiexec -n 3 lb4.exe

2: Hello (p = 3)
2: Bye

0: Hello (p = 3)
0: Process 0 will be do 3333 multiples
0: Process 1 will be do 3333 multiples
0: Process 2 will be do 3334 multiples
0: Total test= 4.961576256972e-03
0: Total=      4.961576256972e-03
0: Time=0.538337 sec.
0: Bye

1: Hello (p = 3)
1: Bye

```

Рисунок 9 - Результат розподілення однакової кількості елементів між процесорами

Далі, для виконання завдання 2.2.3 було виконано порівняння алгоритму який розподіляє елементи згідно кількості операцій, в таблиці – original, та алгоритму, який розподіляє елементи по їх кількості, в таблиці – new. Було виконано запуск обох алгоритмів з N 10000, 20000 та 30000, а також з кількістю процесорів від 1 до 4.

Результати замірів показані на таблиці 4.

Таблиця 4 – Порівняння алгоритмів розподілу елементів

N	P	Original s.	New s.	New/Original
10000	1	0,634358	0,639657	1,008353327
10000	2	0,481438	0,60818	1,263257159
10000	3	0,343679	0,511638	1,488708941
10000	4	0,271673	0,427286	1,572795235
20000	1	4,733578	4,763993	1,006425372
20000	2	2,966941	4,130787	1,392271366
20000	3	2,060964	3,234323	1,569325325
20000	4	1,582009	2,615725	1,653419797
30000	1	12,723431	12,796351	1,005731159
30000	2	7,368556	10,781494	1,46317596
30000	3	5,274115	8,257268	1,565621531
30000	4	3,956555	6,705986	1,69490529

Як можна побачити з таблиці, швидкість виконання при 1 процесорі є однаковою, з похибкою, що означає, що основний алгоритм не був змінений. При збільшенні кількості процесорів алгоритм, який розподіляє елементи згідно кількості операцій є швидшим. Дані таблиці також представленні в графічному форматі на рисунку 10.

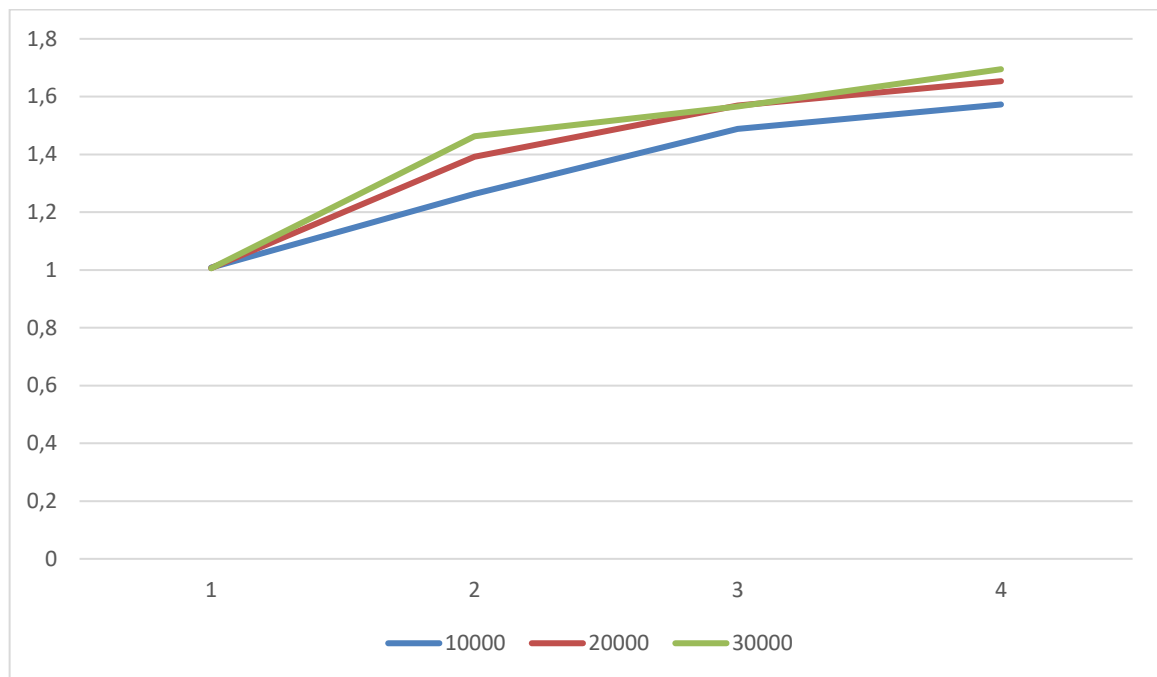


Рисунок 10 – Графік залежності прискорення оригінального алгоритму відносно нового при зміні кількості процесорів та кількості елементів

4 Тест розробленої програми

```
#include "mpi.h"
#include <stdio.h>
#include <math.h>
#include <cstdlib>
#include <ctime>

int first_original(int argc, char** argv) {
    int N = 400000;
    int rank;
    int number_processes;

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &number_processes);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    printf("%d: Hello (p = %d)\n", rank, number_processes);
```

```

double start_time, use_time;

int M;
double* x = nullptr;
double* a = nullptr;

if (rank == 0) {
    x = new double[N];
    a = new double[N];

    srand((unsigned)time(NULL));

    for (long i = 0; i < N; i++) {
        x[i] = -1.073741824 + rand() * 1E-9;
        a[i] = (-1.073741824 + rand() * 1E-9) * 0.1;
    }

    M = N / number_processes;
}

if (rank == 0) {
    start_time = MPI_Wtime();
}

MPI_Bcast(&M, 1, MPI_INT, 0, MPI_COMM_WORLD);

double *xn = new double[M];
double *an = new double[M];

MPI_Scatter(x, M, MPI_DOUBLE, xn, M, MPI_DOUBLE, 0, MPI_COMM_WORLD);
MPI_Scatter(a, M, MPI_DOUBLE, an, M, MPI_DOUBLE, 0, MPI_COMM_WORLD);

double sum = 0.0, total = 0.0;

for (long i = 0; i < M; i++) {
    sum += an[i] * xn[i];
}

MPI_Barrier(MPI_COMM_WORLD);
MPI_Reduce(&sum, &total, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);

if (rank == 0) {
    use_time = MPI_Wtime() - start_time;
    printf("d: t=%lf sec.\n", rank, use_time);
    printf("d: sum in %d procs is %.5f\n", rank, number_processes, total);

    delete[] x;
    delete[] a;
}

delete[] xn;
delete[] an;

MPI_Finalize();
return 0;
}

double cos_by_series(double x) {
    double y = x;
    double s = y;
    int k = 1;
    int K = 500;

    do {
        y = -((x * x) / ((k + 1) * (k + 2))) * y;
    }

```

```

        s = s + y;
        k = k + 2;
    } while (k <= K);

    return s;
}

int first_variant(int argc, char** argv) {
    int rank;
    int number_processes;

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &number_processes);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    printf("\n%d: Hello (p = %d)\n", rank, number_processes);

    int N = 240000;
    if (argc > 1) {
        N = atoi(argv[1]);
    }

    double start_time;
    double use_time;

    double* aFull = nullptr;

    if (rank == 0) {
        aFull = new double[N];

        srand((unsigned)time(NULL));

        printf("%d: Generating array\n", rank);
        for (long i = 0; i < N; i++) {
            aFull[i] = ((double)rand() / RAND_MAX) * 2 - 1;
        }
    }

    if (rank == 0) {
        start_time = MPI_Wtime();
    }

    const int M = N / number_processes;

    double* a = new double[M];

    printf("%d: Received %d elements\n", rank, M);
    MPI_Scatter(aFull, M, MPI_DOUBLE, a, M, MPI_DOUBLE, 0, MPI_COMM_WORLD);

    const double xi = 0.0001;
    double perProcessorSum = 0.0;

    for (long i = 0; i < M; i++) {
        perProcessorSum += a[i] * cos_by_series(xi * i);
    }

    printf("%d: Processor sum = %f\n", rank, perProcessorSum);

    MPI_Barrier(MPI_COMM_WORLD);

    double sum = 0.0;
    MPI_Reduce(&perProcessorSum, &sum, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);

    if (rank == 0) {

```

```

        use_time = MPI_Wtime() - start_time;
        printf("%d: \tN = %d \tP = %d \tT=%lf sec.\n", rank, N, number_processes,
use_time);
        printf("%d: sum in %d procs is %.5f\n", rank, number_processes, sum);
    }

    delete[] aFull;
    delete[] a;

    printf("%d: Goodbye\n", rank);

    MPI_Finalize();
    return 0;
}

int second_original(int argc, char* argv[]) {
    int N = 10000;
    int rank;
    int number_processes;

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &number_processes);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    if (argc > 1) {
        N = atoi(argv[1]);
    }

    // printf("\n%d: Hello (p = %d)\n", rank, number_processes);

    int* process_n = NULL;
    int* process_n_offset = NULL;

    if (rank == 0) {
        process_n = (int*)malloc(number_processes * sizeof(int));

        if (process_n == NULL) {
            printf("%d: Not memory for n\n", rank);
            return -1;
        }

        process_n_offset = (int*)malloc(number_processes * sizeof(int));
        if (process_n_offset == NULL) {
            printf("%d: Not memory for offset\n", rank);
            return -1;
        }

        double perProcess = N * (1.0 + N) / number_processes;
        double an = 1;

        process_n[number_processes - 1] = N;
        for (long i = 0; i < number_processes - 1; i++) {
            double q = (2 * an - 1) / 2.0;
            double result = -q + sqrt(q * q + perProcess);

            an += result;
            process_n[i] = (int)floor(result);
            process_n[number_processes - 1] -= process_n[i];
        }

        process_n_offset[0] = 0;
        for (long i = 1; i < number_processes; i++) {
            process_n_offset[i] = process_n_offset[i - 1] + process_n[i - 1];
        }
    }
}

```

```

        for (long i = 0; i < number_processes; i++) {
            // printf("%d: Process %d will be do %d multiples\n", rank, i,
process_n[i]);
        }

    int n, offset;

    MPI_Scatter(process_n, 1, MPI_INT, &n, 1, MPI_INT, 0, MPI_COMM_WORLD);
    MPI_Scatter(process_n_offset, 1, MPI_INT, &offset, 1, MPI_INT, 0, MPI_COMM_WORLD);

    double* x = NULL;
    double* a = NULL;
    double* xn;
    double* an;

    if (rank == 0) {
        x = (double*)malloc(N * sizeof(*x));
        if (x == NULL) {
            printf("%d: Not memory\n", rank);
            return -1;
        }

        a = (double*)malloc(N * sizeof(*a));
        if (a == NULL) {
            printf("%d: Not memory\n", rank);
            free(x);
            return -1;
        }

        srand((unsigned)time(NULL));

        for (long i = 0; i < N; i++) {
            x[i] = (-1.073741824 + rand() * 1E-9) * 0.8;
            a[i] = (-1.073741824 + rand() * 1E-9) * 0.01;
        }

        xn = (double*)malloc(n * sizeof(*x));
        if (xn == NULL) {
            printf("%d: Not memory\n", rank);
            return -1;
        }

        an = (double*)malloc(n * sizeof(*a));
        if (an == NULL) {
            printf("%d: Not memory\n", rank);
            free(xn);
            return -1;
        }

        double start_time;
        if (rank == 0) {
            start_time = MPI_Wtime();
        }

        MPI_Scatterv(x, process_n, process_n_offset, MPI_DOUBLE, xn, n, MPI_DOUBLE, 0,
MPI_COMM_WORLD);
        MPI_Scatterv(a, process_n, process_n_offset, MPI_DOUBLE, an, n, MPI_DOUBLE, 0,
MPI_COMM_WORLD);

        double processSum = 0.0;

        for (long i = 0; i < n; i++) {
            double mul = 1.0;

```



```

        for (long j = 0; j < i + 1 + offset; j++) {
            mul *= xn[i];
        }

        processSum += an[i] * mul;
        mul = 1.0;
    }

    MPI_Barrier(MPI_COMM_WORLD);

    double sum = 0.0;
    MPI_Reduce(&processSum, &sum, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);

    if (rank == 0) {
        double testSum = 0.0;

        for (long i = 0; i < N; i++) {
            testSum += a[i] * pow(x[i], i + 1);
        }

        // printf("%d: Total test=\t%.12e\n", rank, testSum);
    }

    if (rank == 0) {
        double use_time = MPI_Wtime() - start_time;
        // printf("%d: Total=\t%.12e\n", rank, sum);
        printf("%d: Time=%f sec.\n", rank, use_time);
        free(process_n);
        free(process_n_offset);
    }

    // printf("%d: Bye\n", rank);

    free(x);
    free(a);
    free(xn);
    free(an);
    MPI_Finalize();

    return 0;
}

int second_individual(int argc, char* argv[]) {
    int N = 10000;
    int rank;
    int number_processes;

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &number_processes);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    if (argc > 1) {
        N = atoi(argv[1]);
    }

    // printf("\n%d: Hello (p = %d)\n", rank, number_processes);

    int* process_n = NULL;
    int* process_n_offset = NULL;

    if (rank == 0) {
        process_n = (int*)malloc(number_processes * sizeof(int));

        if (process_n == NULL) {

```

```

        printf("%d: Not memory for n\n", rank);
        return -1;
    }

    process_n_offset = (int*)malloc(number_processes * sizeof(int));
    if (process_n_offset == NULL) {
        printf("%d: Not memory for offset\n", rank);
        return -1;
    }

    int perProcess = N / number_processes;
    for (long i = 0; i < number_processes; i++) {
        process_n[i] = perProcess;
    }
    process_n[number_processes - 1] = perProcess + (N - perProcess *
number_processes);

    process_n_offset[0] = 0;
    for (long i = 1; i < number_processes; i++) {
        process_n_offset[i] = process_n_offset[i - 1] + process_n[i - 1];
    }

    for (long i = 0; i < number_processes; i++) {
        // printf("%d: Process %d will be do %d multiples\n", rank, i,
process_n[i]);
    }

    int n, offset;

    MPI_Scatter(process_n, 1, MPI_INT, &n, 1, MPI_INT, 0, MPI_COMM_WORLD);
    MPI_Scatter(process_n_offset, 1, MPI_INT, &offset, 1, MPI_INT, 0, MPI_COMM_WORLD);

    double* x = NULL;
    double* a = NULL;
    double* xn;
    double* an;

    if (rank == 0) {
        x = (double*)malloc(N * sizeof(*x));
        if (x == NULL) {
            printf("%d: Not memory for x\n", rank);
            return -1;
        }

        a = (double*)malloc(N * sizeof(*a));
        if (a == NULL) {
            printf("%d: Not memory for a\n", rank);
            free(x);
            return -1;
        }

        srand((unsigned)time(NULL));

        for (long i = 0; i < N; i++) {
            x[i] = (-1.073741824 + rand() * 1E-9) * 0.8;
            a[i] = (-1.073741824 + rand() * 1E-9) * 0.01;
        }

        xn = (double*)malloc(n * sizeof(*xn));
        if (xn == NULL) {
            printf("%d: Not memory for xn\n", rank);
            return -1;
        }
    }

```

```

an = (double*)malloc(n * sizeof(*a));
if (an == NULL) {
    printf("%d: Not memory for an\n", rank);
    free(xn);
    return -1;
}

double start_time;
if (rank == 0) {
    start_time = MPI_Wtime();
}

MPI_Scatterv(x, process_n, process_n_offset, MPI_DOUBLE, xn, n, MPI_DOUBLE, 0,
MPI_COMM_WORLD);
MPI_Scatterv(a, process_n, process_n_offset, MPI_DOUBLE, an, n, MPI_DOUBLE, 0,
MPI_COMM_WORLD);

double processSum = 0.0;

for (long i = 0; i < n; i++) {
    double mul = 1.0;

    for (long j = 0; j < i + 1 + offset; j++) {
        mul *= xn[i];
    }

    processSum += an[i] * mul;
    mul = 1.0;
}

MPI_Barrier(MPI_COMM_WORLD);

double sum = 0.0;
MPI_Reduce(&processSum, &sum, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);

if (rank == 0) {
    double testSum = 0.0;

    for (long i = 0; i < N; i++) {
        testSum += a[i] * pow(x[i], i + 1);
    }

    // printf("%d: Total test=\t%.12e\n", rank, testSum);
}

if (rank == 0) {
    double use_time = MPI_Wtime() - start_time;
    // printf("%d: Total=\t%.12e\n", rank, sum);
    printf("%d: \t N = %d \t p = %d \t Time=%f sec.\n", rank, N, number_processes,
use_time);
    free(process_n);
    free(process_n_offset);
}

// printf("%d: Bye\n", rank);

free(x);
free(a);
free(xn);
free(an);
MPI_Finalize();

return 0;
}

```

```
int main(int argc, char** argv) {  
    second_original(argc, argv);  
    // second_individual(argc, argv);  
}
```

5 Висновки

Я продовжив вивчення функцій обміну бібліотеки MPI, зокрема, функцій колективного обміну. Освоїв деякі прийоми їх використання для розподілу даних та обчислень між паралельними процесорами